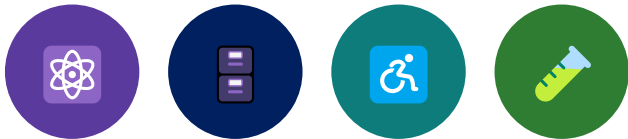


MODULE 50

Capstone Frontend and Persistence

Complete the Northstar CRM agent journey: an accessible React search, profile, timeline, and interaction form, backed by durable PostgreSQL persistence via Spring Data JPA, consuming Module 49's API contracts exactly.







MODULE 50 OVERVIEW

Complete a usable React CRM journey backed by durable PostgreSQL persistence via Spring Data JPA, consuming Module 49's API exactly.

Agents need an accessible search, profile, timeline, and interaction form with durable PostgreSQL storage. A UI that looks done without database proof or accessibility basics fails the capstone gate.

DURATION & LAB



4 Hours Theory

React review, UI development, backend integration, PostgreSQL/Oracle comparison, JPA, and testing.



2 Hours Lab

lab50-crm: React journey, migrations, JPA mappings, typed client, a11y, restart-durability proof.



Scenario

Search Amina (CUS-1001), open her timeline, record an interaction with lab-request-001.

MODULE ARC



Journey First

Search, profile, timeline, form -- one coherent flow, not a component zoo.



PostgreSQL Proof

A toast is not persistence -- prove the row survives an API restart.



Accessibility Is Graded

Labels, keyboard order, and role=alert announcements are rubric marks, not polish.

KEY TAKEAWAY Total time: 4 hours theory + 2 hours lab = 6 hours. No pass without UI proof, PostgreSQL row proof, accessibility basics, and one automated UI/component check.

WHY FULL-STACK INTEGRATION MATTERS

The frontend must prove real persistence and real accessibility -- not a component-library demo with no database behind it.



A Journey, Not Widgets

Search, profile, timeline, and form must connect into one usable agent flow.



Pretty Without Proof Fails

A polished UI with no SQL row or restart-durability proof loses core implementation marks.



Accessibility Is Not Optional

Missing labels or keyboard support is a named security/production-awareness deduction.

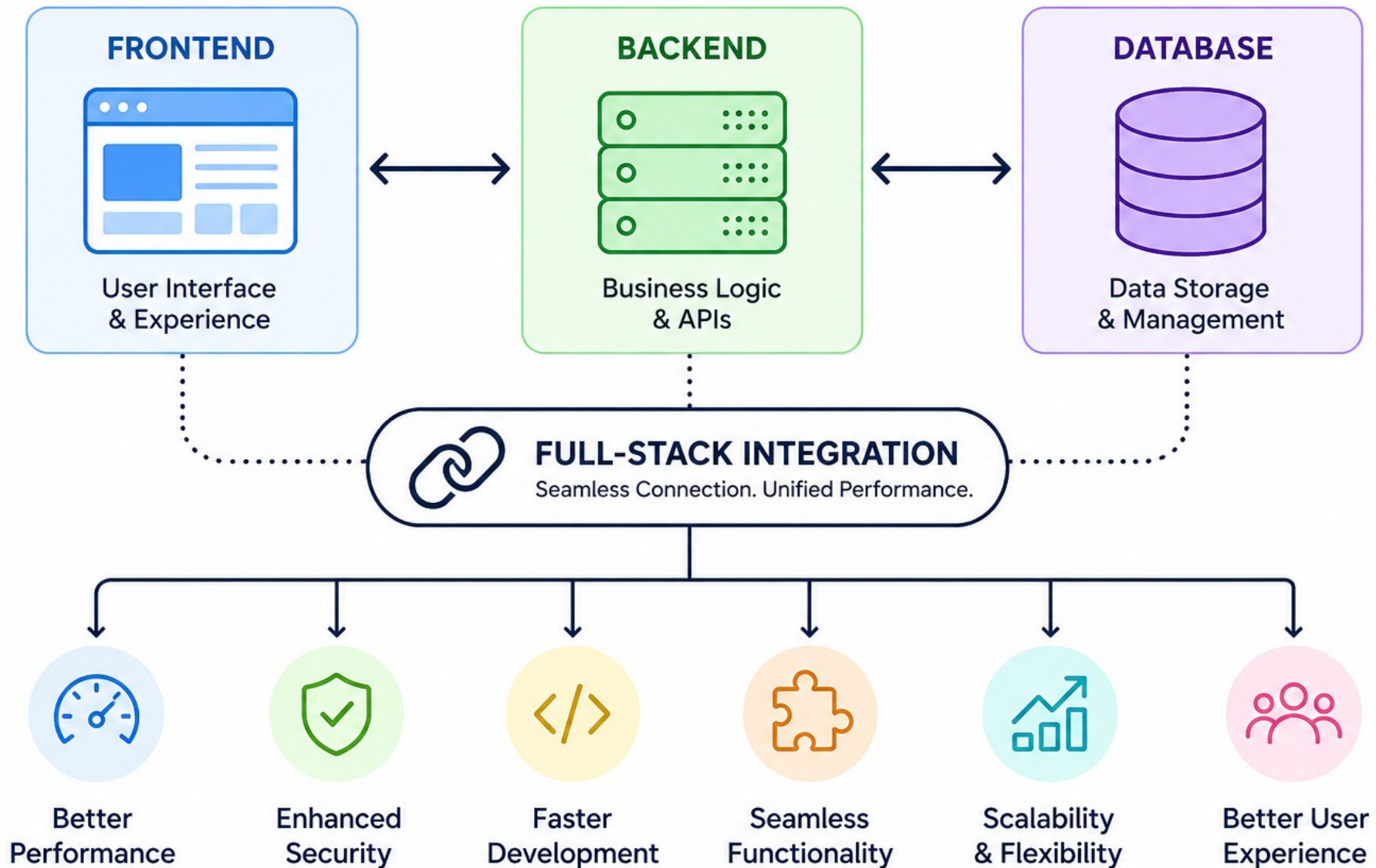


Graded Like Every Lab

Lab 50 is scored on the same 100-point rubric: core impl, integration, failure handling, verification, security/a11y, docs.

KEY TAKEAWAY Pretty UI without SQL proof is an incomplete core-implementation mark -- missing labels or keyboard support costs accessibility and security marks too.

Why Full-Stack Integration Matters



MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to build and prove a full-stack CRM journey for the Northstar CRM.

1. Build typed React service calls against Module 49's API contracts.
2. Implement accessible forms with labels, focus management, and alerts.
3. Complete JPA and PostgreSQL mappings with deliberate relationship ownership.
4. Create repeatable, versioned PostgreSQL migrations.
5. Handle loading, empty, success, invalid, unauthorized, and outage states.
6. Verify UI-to-database flow with restart durability.
7. Automate component tests and one critical-path E2E journey.
8. Document a reproducible frontend-and-persistence demo for Module 52.

KEY TAKEAWAY These eight objectives are drawn directly from the graded lab -- you will build and prove every layer of the search-to-interaction journey.

REACT ARCHITECTURE REVIEW

A quick review of the React shape this module builds on -- components, state, and a typed client, wired through props and hooks.



Component Tree

CustomerSearch, CustomerProfile, InteractionTimeline, InteractionForm compose into App.



Unidirectional Data Flow

State lives high enough to be shared; props flow down, events flow up.



Hooks for Server State

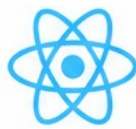
useState/useEffect (or an equivalent data layer) drive loading/error/data transitions.



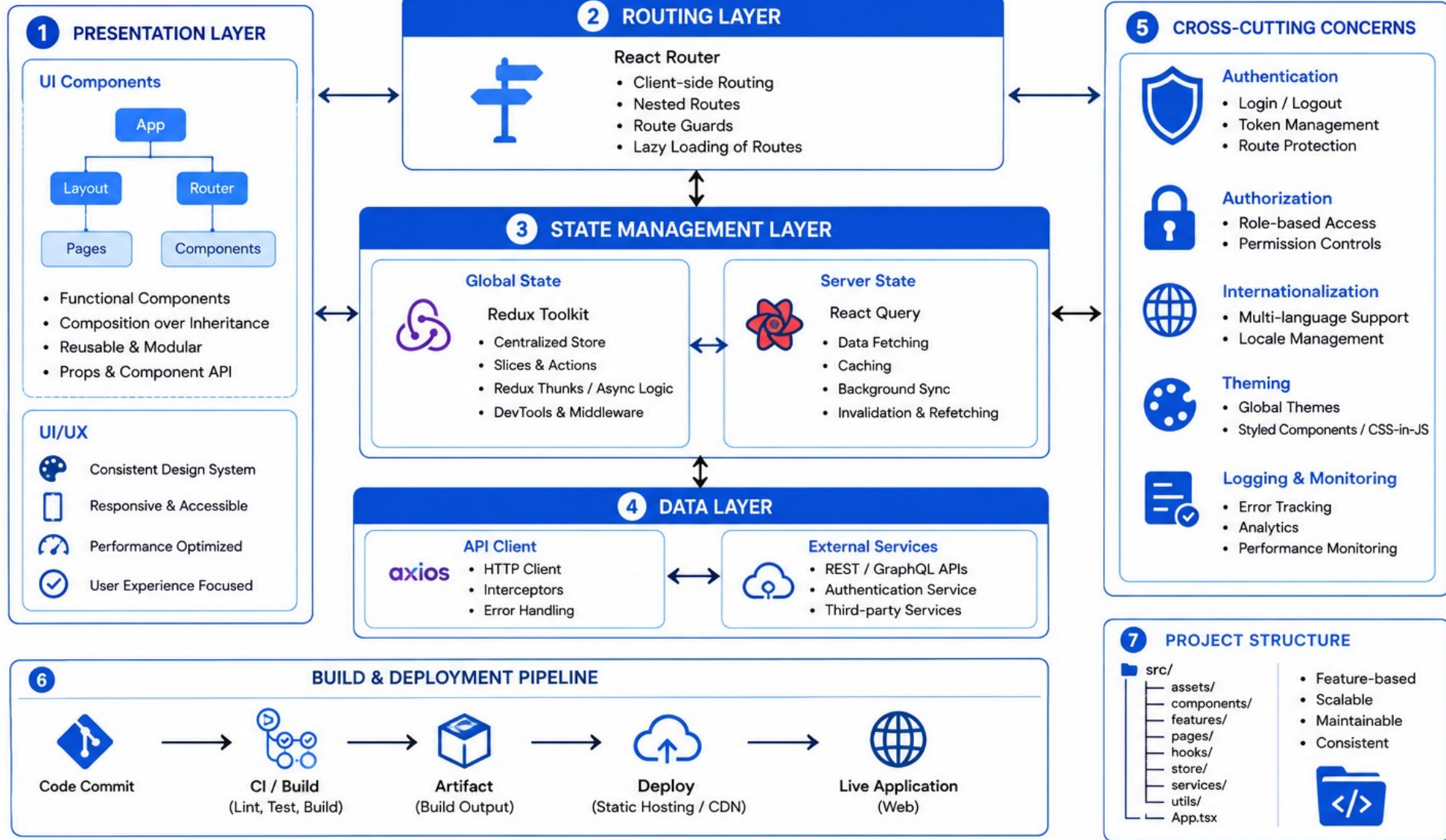
Vite/Node 22 Build

The same frontend toolchain from earlier weeks, now pointed at the Lab 49 API.

KEY TAKEAWAY Everything built this module reuses the same component and state shape from earlier weeks -- applied now to a real capstone journey.



REACT ARCHITECTURE REVIEW



KEY PRINCIPLES



Component Reusability



Single Source of Truth



Separation of Concerns



Scalability



Maintainability

COMPONENT DESIGN REVIEW

Four components, four jobs -- search, profile, timeline, and form each own one responsibility and hand off cleanly.

COMPONENT RESPONSIBILITIES

CustomerSearch: query input, results list, empty/loading states.

CustomerProfile: header details -- name, id, status badge.

InteractionTimeline: read model, newest first, refreshed after writes.

InteractionForm: the one write path, wired to the typed client.

WHAT EACH COMPONENT AVOIDS

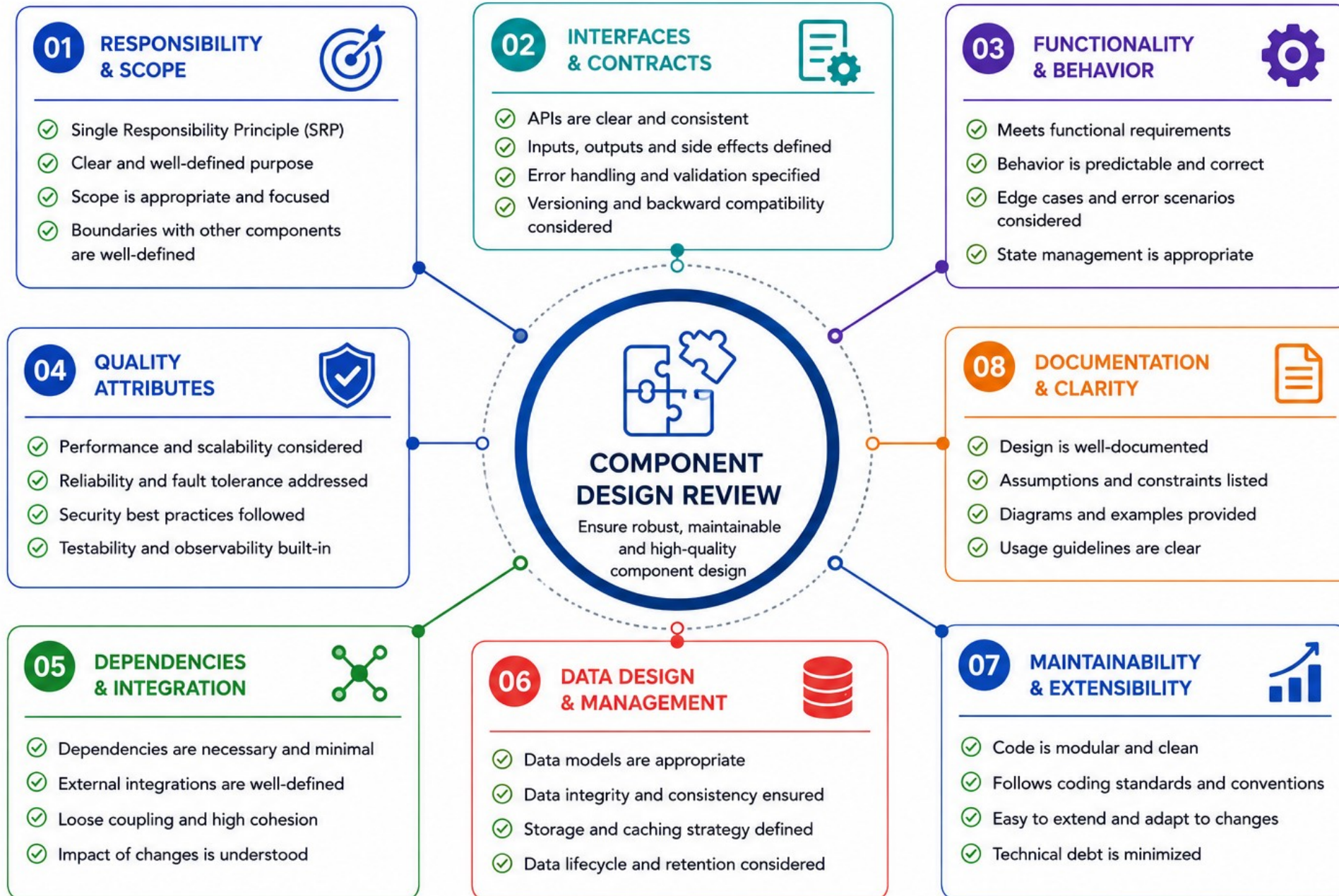
- Search avoids embedding profile-rendering logic.
- Profile avoids fetching interactions itself -- the timeline owns that.
- Timeline avoids duplicating the form's validation rules.
- No component calls fetch directly -- only the typed client does.

Composition (excerpt)

```
<App>
  <CustomerSearch onSelect={setCustomerId} />
  {customerId && <CustomerProfile id={customerId} />}
  {customerId && <InteractionTimeline customerId={customerId} refreshKey={key} />}
  {customerId && <InteractionForm customerId={customerId} onSave={bumpKey} />}
</App>
```

KEY TAKEAWAY A component that fetches data outside the typed client, or duplicates another component's job, is exactly the layering mistake this review exists to prevent.

COMPONENT DESIGN REVIEW



STATE MANAGEMENT REVIEW

Server state and UI state are different problems -- keep them separate so loading, error, and success are never guessed at.

TWO KINDS OF STATE

Server state: customer, timeline, and form-submission results.

UI state: which screen is active, form field values, validation messages.

Selected customer id lives above profile and timeline (shared).

Submitting flag disables the form's submit button while in flight.

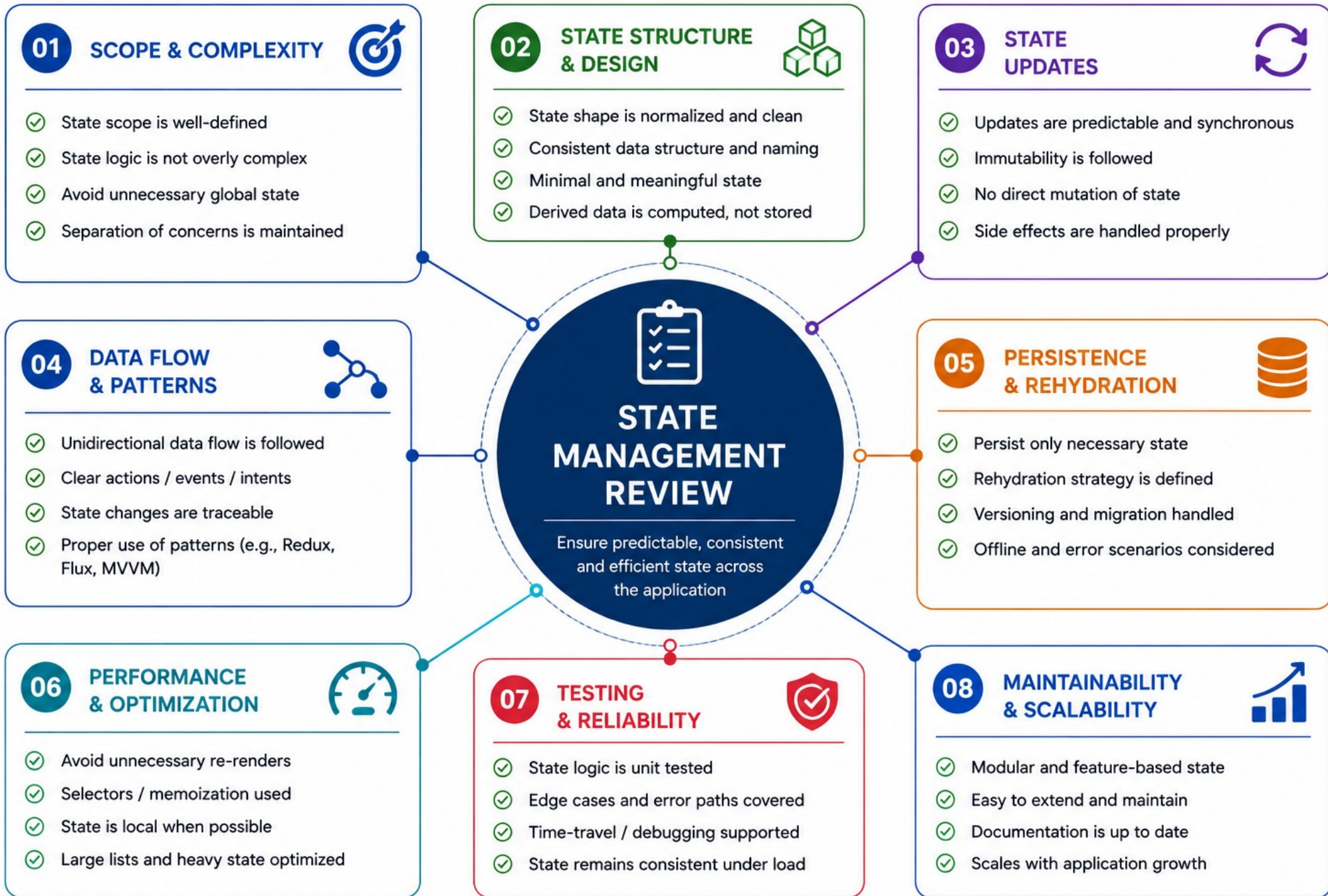
STATE DISCIPLINE

- Never treat an in-flight request as already succeeded.
- Refresh (refetch) server state after a write -- do not hand-patch it.
- One source of truth per piece of state -- no duplicated copies drifting apart.
- Stale responses from a superseded request must be ignored.

Submitting-state guard (concept)

```
const [submitting, setSubmitting] = useState(false);
async function onSubmit(data) {
  if (submitting) return; // guard against double submit
  setSubmitting(true);
  try { await createInteraction(customerId, data); onSave(); }
  finally { setSubmitting(false); } }
```

KEY TAKEAWAY A double-submit that creates two rows, or a stale response overwriting a fresher one, are both state-management bugs this review exists to prevent.



API INTEGRATION REVIEW

A quick review of how the React client talks to Module 49's Spring Boot API -- typed, centralized, and cancellable.

Term	Meaning
Typed client module	One place that knows the base URL, headers, and response shapes.
AbortController	Cancels an in-flight request when a component unmounts or a newer one starts.
Correlation header	X-Correlation-ID: lab-request-001 sent on every write, matching Module 49.
Problem Details	The 400/404 JSON error shape from Module 49's exception handling.

WHY CENTRALIZE THE CLIENT



One Base URL

Every component imports the same client -- no hardcoded hosts.



Typed Requests/Responses

TypeScript interfaces mirror Module 49's DTOs field for field.



Cancellation-Safe

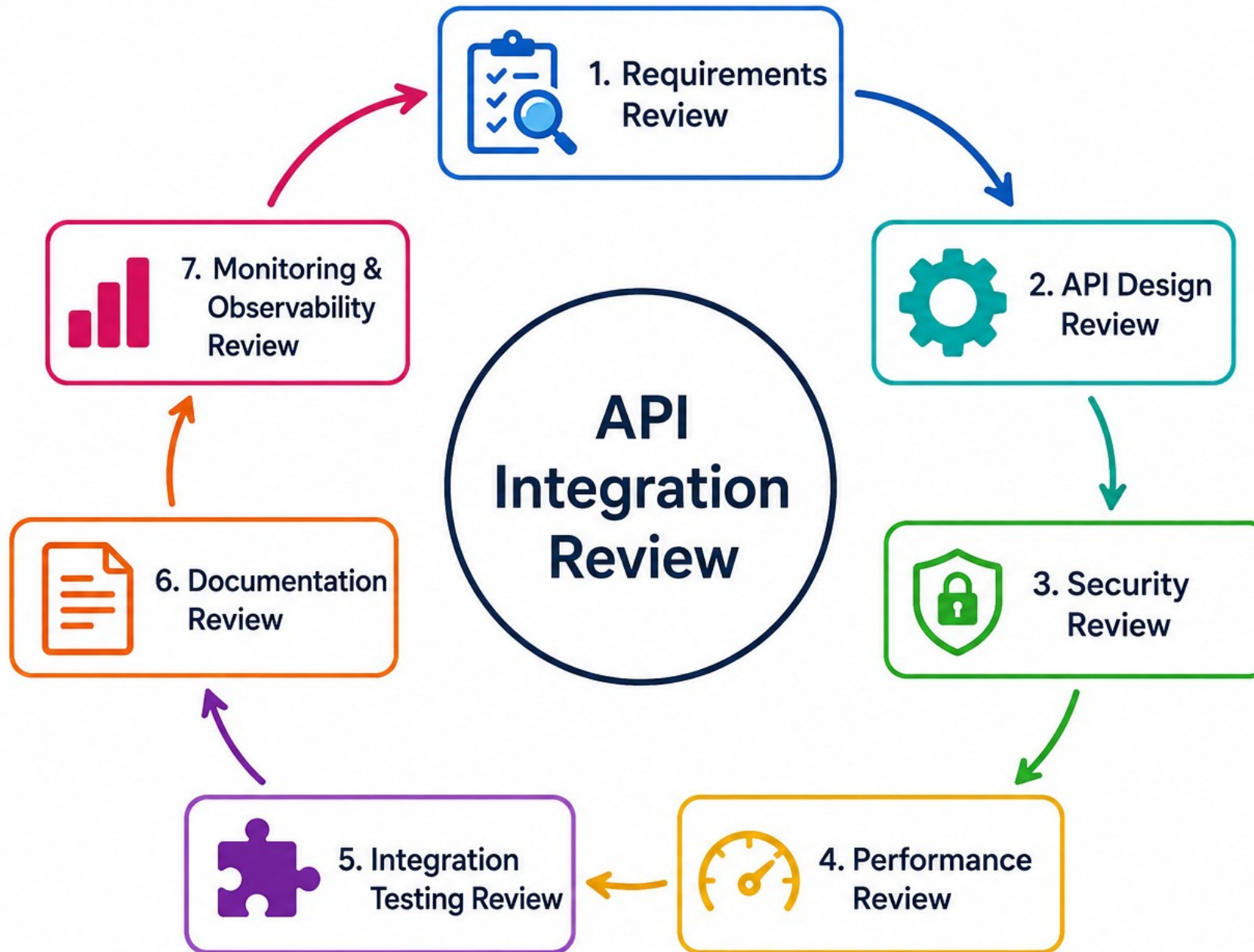
AbortSignal prevents a slow, superseded request from winning a race.



Correlation Always Sent

lab-request-001 on writes -- traceable end to end with Module 49's logs.

KEY TAKEAWAY An ad-hoc fetch copy-pasted into a component drifts from Module 49's contract the moment that contract changes -- a single typed client is the only source of truth.



TRY IT YOURSELF -- EXERCISE 1: MAP AGENT UI JOURNEY

Reinforces: storyboarding the full search-profile-timeline-form journey with fixtures and per-screen states, before any component code is written.

STEPS (notes/lab50-journey-map.md)

Step	What To Do
1 -- Screens	Search, profile header, interaction timeline, add-interaction form.
2 -- Check the Reference	Journey must be usable end to end -- not a disconnected component zoo.
3 -- Fixtures	Plan demo data: search finds CUS-1001 Amina; CUS-1002 Ravi is optional.
4 -- States	For each screen, note loading/empty/error placeholders.

Journey map

```
Screens: Search -> Profile -> Timeline -> Form.  
Fixtures: CUS-1001 (Amina), CUS-1002 (Ravi).  States: loading/empty/error per screen.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 1 before continuing: exercises/exercise-01-journey-map.md (workspace: examples/module-50-exercises).

DASHBOARD DESIGN

The landing view orients an agent fast -- search front and center, with clear entry into a customer's profile.

DASHBOARD ELEMENTS

Search box is the primary, above-the-fold action.

Recent or suggested customers, if scope allows.

Clear visual state for 'no customer selected yet'.

Status badges (ACTIVE/PROSPECT) readable at a glance.

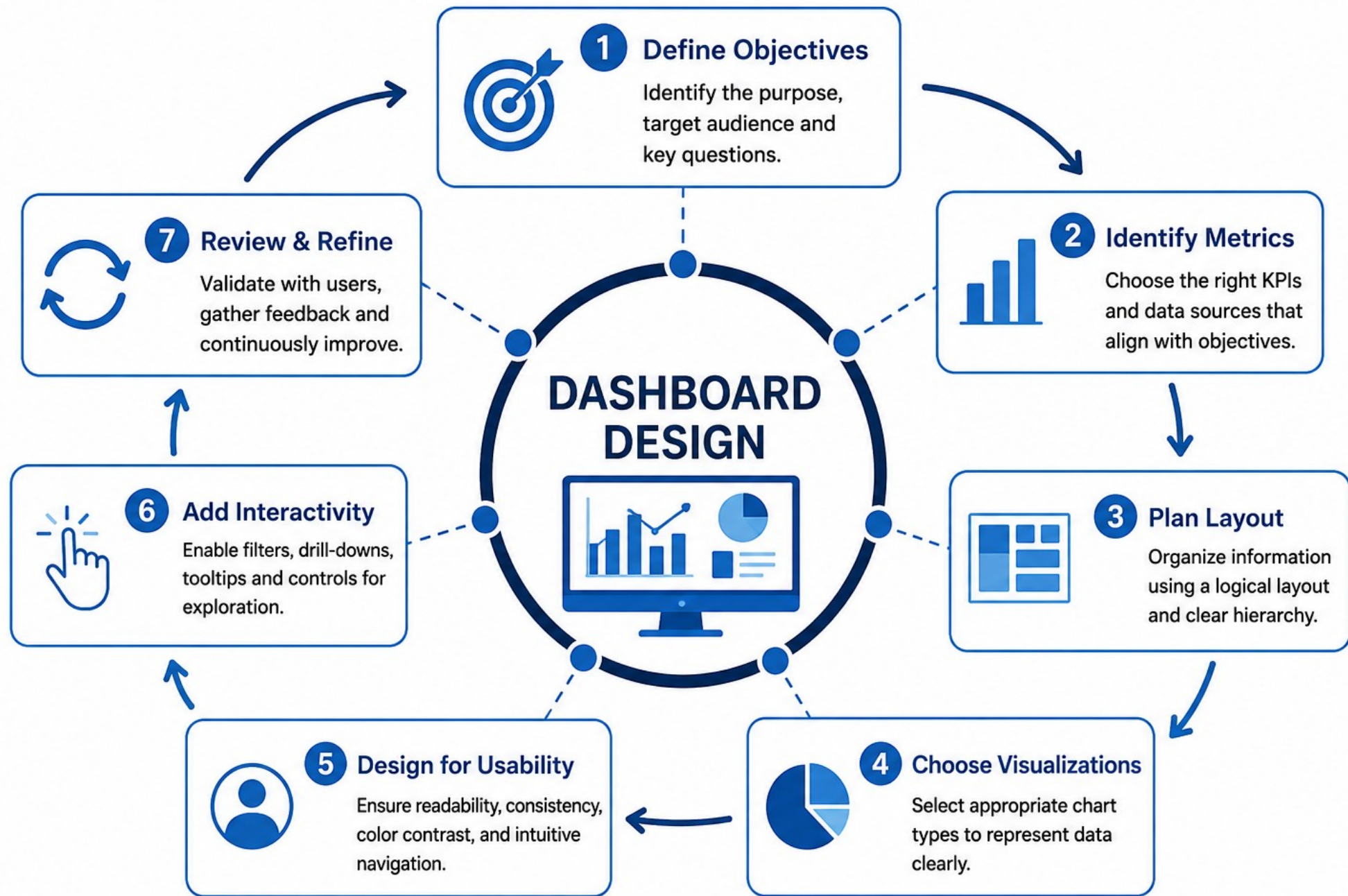
DESIGN DISCIPLINE

- Do not bury search behind a menu or a second click.
- Keep the layout usable at common laptop widths.
- Landmarks (main, nav) over generic div soup for screen readers.
- No customer data rendered until a real search resolves.

Landmark structure (excerpt)

```
<main>
  <h1>Northstar CRM</h1>
  <CustomerSearch onSelect={setCustomerId} />
  {!customerId && <p>Search for a customer to begin.</p>}
</main>
```

KEY TAKEAWAY An agent under call-handling time pressure should reach search within one glance and one click -- design the dashboard around that constraint.



CUSTOMER MANAGEMENT SCREENS

Profile and timeline together tell the agent everything about a customer without a second navigation click.

PROFILE + TIMELINE

Profile: name, customer id, status badge, key contact fields.

Timeline: interactions newest first, channel and summary visible.

Timeline refreshes automatically after a successful create.

Empty timeline shows an explicit message, not a blank panel.

SCREEN DISCIPLINE

- Profile never fetches interactions -- that stays the timeline's job.
- No JPA entity field names ever leak into the UI verbatim.
- CUS-9999 must render a clear not-found state, not a crash.
- Screen reader users must reach both regions via landmarks/headings.

Timeline render (concept)

```
{interactions.length === 0
  ? <p>No interactions recorded yet.</p>
  : interactions.map(i => <TimelineItem key={i.id} {...i} />)}
```

KEY TAKEAWAY A profile screen that silently shows nothing for CUS-9999 fails the not-found empty state Lab 50 explicitly grades.

Customer Management Screens

1 Customer List

Dashboard

Customers

Add Customer

Contacts

Activities

Reports

Settings

Customers

Search customers...

Filters

Customer ID	Customer Name	Email	Phone	Status	Actions
CUST-1001	Acme Corporation	info@acme.com	(555) 123-4567	Active	
CUST-1002	Globex Inc.	contact@globex.com	(555) 234-5678	Active	
CUST-1003	Initech	hello@initech.com	(555) 345-6789	Inactive	
CUST-1004	Umbrella Corp.	support@umbrella.com	(555) 456-7890	Active	
CUST-1005	Stark Industries	info@stark.com	(555) 567-8901	Active	

Showing 1 to 5 of 25 entries

1

2

3

4

5

2 Customer Details

Back to List

Edit

Acme Corporation

Active

Customer ID

CUST-1001

Email

info@acme.com

Phone

(555) 123-4567

Overview

Contacts

Addresses

Activities

Notes

Files

Company Information

Billing Information

Customer Type

Corporate

Industry

Manufacturing

Website

www.acme.com

Tax ID

12-3456789

Created On

Jan 15, 2024

Customer Since

Jan 15, 2024

Currency

USD

Payment Terms

Net 30

Credit Limit

\$50,000.00

Outstanding

\$12,450.00

3 Add / Edit Customer

Back to List

Cancel

Save

Customer Information

Customer Name *

Acme Corporation

Customer Type *

Corporate

Email *

info@acme.com

Phone *

(555) 123-4567

Industry

Manufacturing

Website

www.acme.com

Tax ID

12-3456789

Status

Active

Billing Information

Currency

USD

Payment Terms

Net 30

Credit Limit

50000.00

Notes

Preferred customer.

6 Customer Reports

Reports

Customer Summary

Sales by Customer

Outstanding Invoices

Customer Aging

Top Customers

New Customers

Activity Summary

Total Customers

125

Active Customers

98

Inactive Customers

27

Customers by Status

Active 78% (98)

Inactive 22% (27)

Customers by Industry

Manufacturing (40)

Technology (30)

Retail (25)

Other (30)

Top Customers by Revenue

Customer Name	Revenue (USD)
Acme Corporation	\$125,000.00
Globex Inc.	\$98,500.00
Stark Industries	\$75,000.00
Umbrella Corp.	\$60,750.00
Initech	\$45,250.00

5 Customer Activities

Back to Customer Details

Add Activity

Activities

Date	Type	Subject	Assigned To	Status
May 01, 2024	Call	Quarterly Follow-up	Jennifer Lee	Completed
Apr 28, 2024	Email	Product Information	Michael Chen	Completed
Apr 20, 2024	Meeting	Requirement Discussion	Jennifer Lee	Completed
Apr 10, 2024	Task	Pricing Proposal	Michael Chen	Pending
Apr 05, 2024	Call	Introductory Call	Jennifer Lee	Completed

Showing 1 to 5 of 12 entries

1

2

3

4 Customer Contacts

Back to Customer Details

Add Contact

Contacts

Contact Name	Designation	Email	Phone	Actions
John Smith	Sales Manager	john.smith@acme.com	(555) 111-2222	
Mary Johnson	Account Manager	mary.j@acme.com	(555) 222-3333	
Robert Brown	Support Lead	robert.b@acme.com	(555) 333-4444	

Showing 1 to 5 of 3 entries

SEARCH AND FILTER FUNCTIONALITY

Search must find Amina and Ravi reliably, and behave predictably when nothing matches.

SEARCH BEHAVIOR

Search by name or customer id resolves CUS-1001 and CUS-1002.

Debounce or explicit submit -- avoid firing a request per keystroke.

Loading indicator while a search request is in flight.

Empty-result state is distinct from the not-found single-customer state.

INPUT DISCIPLINE

- The search input has a real, visible label -- never placeholder-as-label.
- Enter key submits the search; button remains keyboard-activatable.
- Results are keyboard-navigable, not mouse-only.
- No client-side filter silently hides a customer the API actually returned.

Search submit (concept)

```
<label htmlFor="q">Search customers</label>
<input id="q" name="q" value={q} onChange={e => setQ(e.target.value)} />
<button type="submit">Search</button>
```

KEY TAKEAWAY A search box with a placeholder instead of a real label passes a visual glance but fails the keyboard and screen-reader checklist every time.

Search and Filter Functionality

Find the right data quickly using search, filters, and saved views.

1 Search Bar

Search across customer name, ID, email, phone, address and more. Real-time results as you type.

2 Filters

Narrow down results using multiple filter criteria. Combine filters for precise data.

4 Active Filters

View applied filters as chips. Easily remove individual filters or clear all.

3 Saved Views

Save frequently used filter combinations as views for quick access. Manage and switch between views.

5 Actions

Perform quick actions on each record: View, Edit or More options.

6 Pagination

Navigate through results with easy pagination controls and total count.

The screenshot shows a web application interface for managing customers. On the left is a dark sidebar with navigation links: Home, Dashboard, Customers (selected), Contacts, Orders, Invoices, Reports, and Settings. The main content area is titled 'Customers' and includes a '+ Add Customer' button. Below the title is a search bar (labeled 1) and a filters section (labeled 2) containing chips for 'Status: Active', 'Industry: Manufacturing', and 'Country: USA'. A 'Clear All' button is next to the filters. To the right of the filters is a 'Saved Views' dropdown (labeled 3) and a 'Save View' button. The central part of the interface is a table with columns: Customer ID, Customer Name, Industry, Country, Status, Created Date, and Actions. The table contains six rows of customer data. At the bottom of the table, it says 'Showing 1 to 6 of 42 entries' and a pagination control (labeled 6) with buttons for '<', '1', '2', '3', '4', '5', and '>'. The '1' button is highlighted.

☐	Customer ID	Customer Name	Industry	Country	Status	Created Date	Actions
☐	CUST-1001	Acme Corporation	Manufacturing	USA	Active	May 15, 2024	
☐	CUST-1002	Globex Inc.	Technology	USA	Active	May 14, 2024	
☐	CUST-1003	Initech	Manufacturing	USA	Inactive	May 13, 2024	
☐	CUST-1004	Umbrella Corp.	Pharmaceuticals	USA	Active	May 12, 2024	
☐	CUST-1005	Stark Industries	Technology	USA	Active	May 11, 2024	
☐	CUST-1006	Wayne Enterprises	Manufacturing	USA	Active	May 10, 2024	

The screenshot shows a 'Filter Panel (Example)' with several filter groups. Each group has a dropdown menu and a 'Cancel' button. The groups are: Status (Active), Industry (Manufacturing), Country (USA), Created Date (Custom Range), Customer Type (All), Account Manager (All), Tags (Select tags), and Date Range (May 01, 2024 - May 31, 2024). At the bottom right are buttons for 'Cancel', 'Clear All', and 'Apply Filters'.

Filter Panel (Example)

Status: Active
Industry: Manufacturing
Country: USA
Created Date: Custom Range
Customer Type: All
Account Manager: All
Tags: Select tags
Date Range: May 01, 2024 - May 31, 2024

Buttons: Cancel, Clear All, Apply Filters

FORM DESIGN BEST PRACTICES

InteractionForm is the one write path in this journey -- every label, error, and focus decision here is graded.

FORM STRUCTURE

Real <label htmlFor> for every field -- never placeholder-only.

channel and summary map exactly to Module 49's CreateInteractionRequest.

aria-describedby links help text and error text to the field.

Submit button disables while submitting; text changes to 'Saving...'.

VALIDATION AND FEEDBACK

- Client-side checks are UX help, not the security boundary (the API still validates).
- role="alert" announces errors to assistive technology immediately.
- Field-level Problem Details map to field-level messages, not one generic banner.
- Focus moves to a sensible place after a successful save.

Accessible field (excerpt, from the lab guide)

```
<label htmlFor="summary">Interaction summary</label>
<textarea id="summary" name="summary" required maxLength={1000}
  aria-describedby="summary-help summary-error" />
{error && <p id="summary-error" role="alert">{error}</p>}
```

KEY TAKEAWAY A modal or form that traps keyboard focus forever, or an error only shown in color, fails the keyboard-only pass this module explicitly requires.



TRY IT YOURSELF -- EXERCISE 4: FILL ACCESSIBILITY TODOS

Reinforces: completing a concrete accessibility checklist for the search box, form, and timeline before implementation.

STEPS (notes/lab50-a11y-todos.md)

Step	What To Do
1 -- Template	Fill blanks: form labels, error announced, keyboard path, contrast, focus on error.
2 -- Search Box	Confirm the search input has a visible label/name for agents.
3 -- Timeline	Note how an empty timeline is announced vs. a silent blank panel.
4 -- Scope	Checklist warmup only -- implementation happens during Lab 50.

A11y checklist skeleton

```
Form labels: real <label htmlFor>.  Error announced: role="alert".  
Keyboard path: tab search -> results -> profile -> form.  Focus on submit error: kept on field.
```

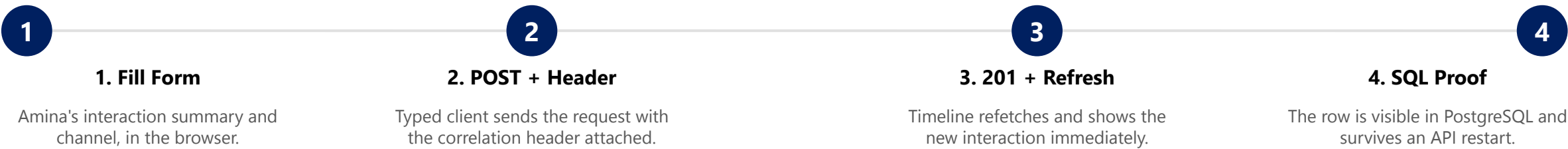
KEY TAKEAWAY TRY IT NOW -- Complete Exercise 4 before continuing: exercises/exercise-04-a11y-todos.md (workspace: examples/module-50-exercises).

API CONSUMPTION WORKFLOW

One user action, four checkpoints -- from a form submit to a durable, visible timeline update.

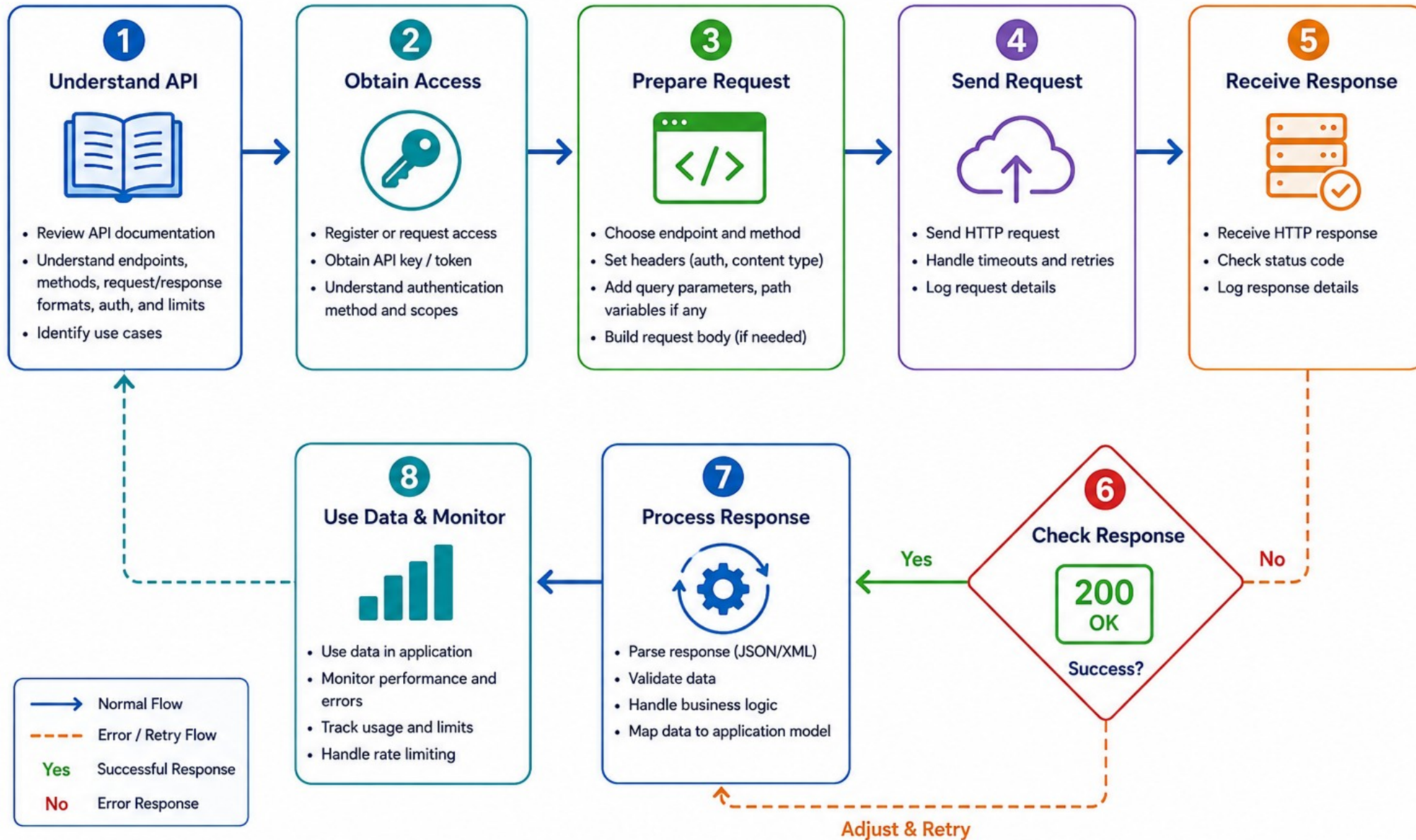
Stage	What Happens	Verified By
1. Validate (client)	Required fields checked before the request is even sent.	Component test, empty summary
2. Call (typed client)	POST with X-Correlation-ID: lab-request-001, Content-Type json.	Network tab / test spy
3. Handle response	201 refreshes the timeline; 400/404/401 map to visible messages.	Component test per status
4. Persist (backend)	Module 49's service saves inside its own transaction.	Lab 49 evidence + SQL row

END-TO-END TRACE (lab-request-001)



KEY TAKEAWAY A UI that shows success on a 400 response, or never refreshes the timeline after a real 201, breaks this workflow at exactly the checkpoint meant to catch it.

API CONSUMPTION WORKFLOW



TRY IT YOURSELF -- EXERCISE 2: PLAN TYPED API CLIENT

Reinforces: listing the client functions and error mapping the journey needs, before writing any fetch code.

STEPS (notes/lab50-api-client-plan.md)

Step	What To Do
1 -- Functions	searchCustomers, getCustomer, listInteractions, createInteraction (adaptable names).
2 -- Check the Reference	Typed calls reduce silent UI breakage when the API evolves.
3 -- Error Mapping	Map HTTP 401/403/404/500 to user-visible messages -- no stack traces.
4 -- Auth Header	Note where a JWT will attach later -- never hardcode a token in source.

Client function reference

```
loading -> spinner/skeleton.    empty -> clear guidance.  
error -> actionable message.    success -> data or confirmation.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 2 before continuing: exercises/exercise-02-api-client-plan.md (workspace: examples/module-50-exercises).

ERROR HANDLING IN REACT

Problem Details from Module 49 becomes a specific, readable message in the UI -- never a generic failure screen.

MAPPING PROBLEM DETAILS

response.ok check happens before any optimistic UI update.

400 detail/fieldErrors map to field-level alerts.

404 (CUS-9999) maps to a clear not-found message.

401/403 map to a sign-in or access-denied message, not a crash.

ERROR-HANDLING DISCIPLINE

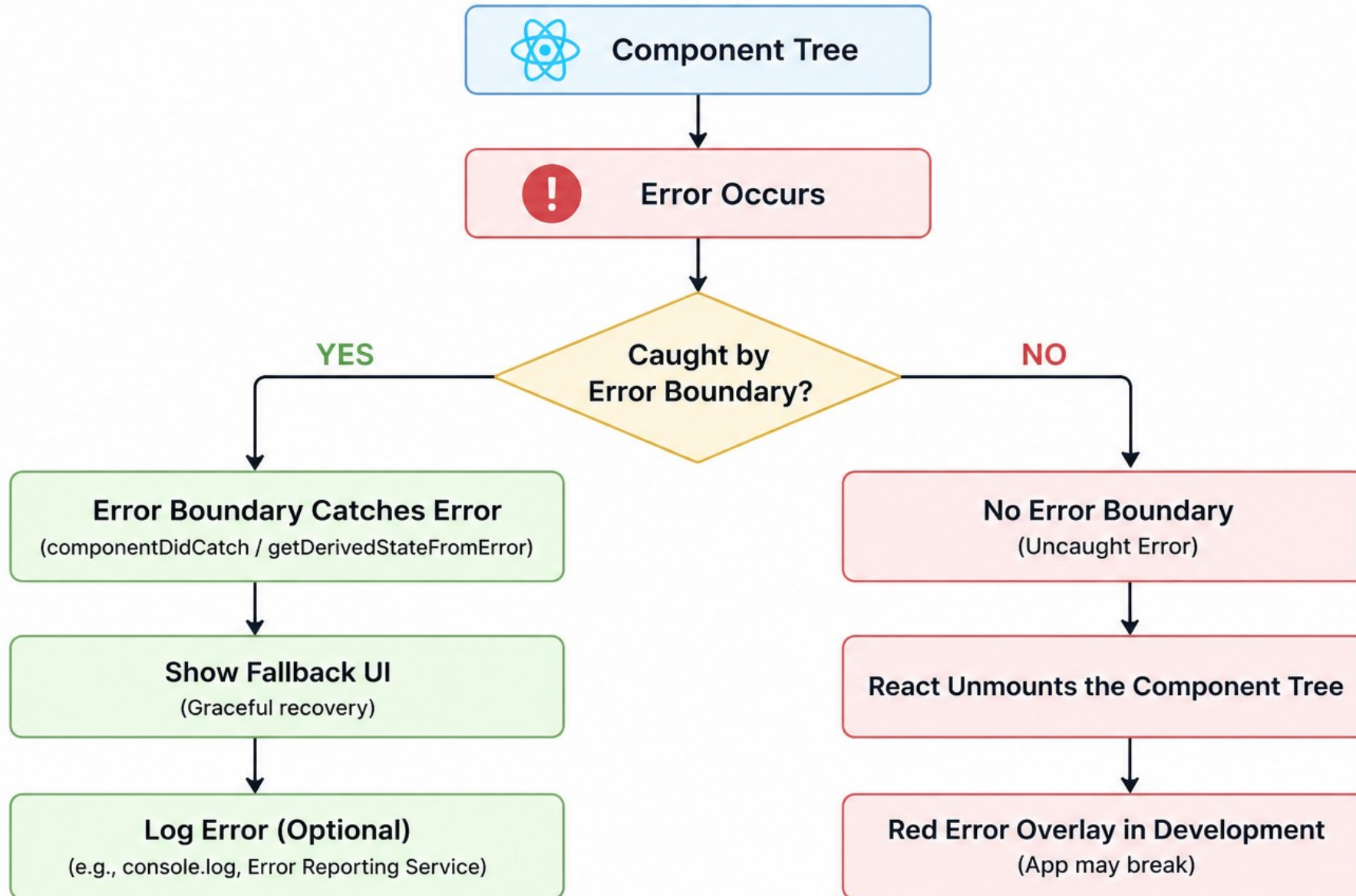
- Never show a success toast before checking response.ok.
- Never leak a raw stack trace or exception message to the UI.
- Distinguish invalid input (400) from an outage (network/5xx).
- Log enough to debug -- never log a bearer token to the console.

Response handling (concept)

```
if (!response.ok) throw await toApiError(response);  
// toApiError reads Problem Details JSON, never a raw exception string
```

KEY TAKEAWAY A success toast rendered before response.ok is checked is the exact bug this module's failure experiments are designed to catch.

Error Handling in React

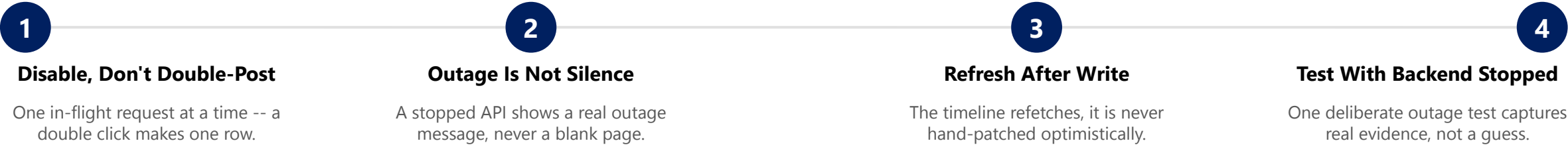


LOADING AND FEEDBACK PATTERNS

Six states, six behaviors -- happy-path-only UIs lie to the agent the moment PostgreSQL or the API has a bad day.

State	Trigger	UI Behavior
Loading	In-flight GET/POST	Spinner or skeleton; submit disabled.
Empty	No interactions yet	Explicit empty message, not a blank panel.
Success	201 response	Timeline refresh; brief confirmation.
Invalid	400 response	role="alert" field errors.
Unauthorized	401/403 response	Sign-in or access message.
Outage	Network error or 5xx	Retry guidance -- never a fake success.

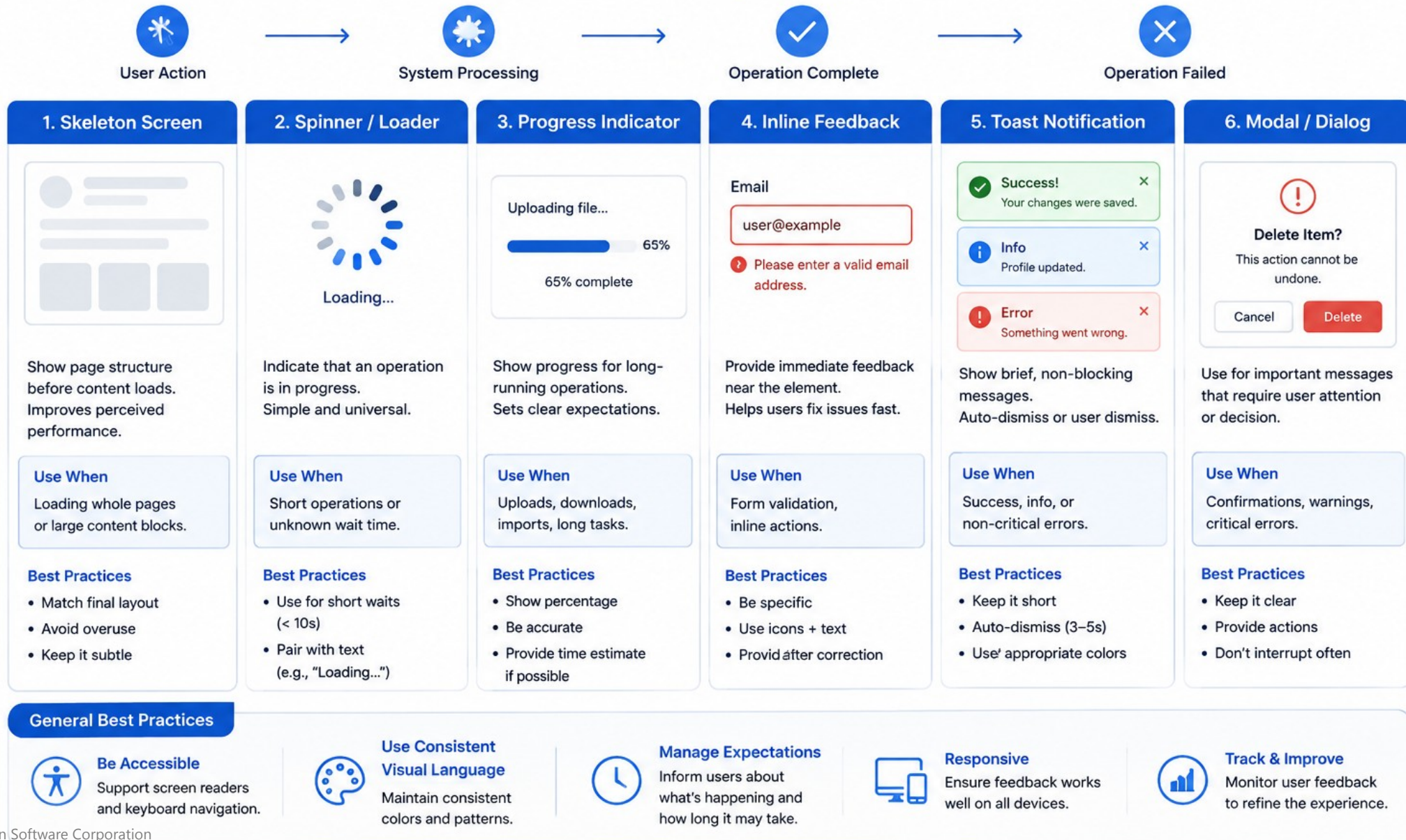
STATE-HANDLING HABITS



KEY TAKEAWAY Test with the backend stopped once, on purpose -- capturing real outage UI evidence beats assuming it 'probably works'.

Loading and Feedback Patterns

Provide clear feedback to users during async operations and interactions



DATABASE SCHEMA REVIEW

customer_interaction already exists from Module 49 -- this module reviews it against PostgreSQL-specific types and constraints.

SCHEMA ELEMENTS

interaction_id, customer_id, channel, summary, created_at, version.

Primary key on interaction_id; foreign key to customer.

CHECK constraint restricts channel to PHONE/EMAIL/CHAT.

Index on (customer_id, created_at) for the timeline query.

REVIEW DISCIPLINE

- Confirm types are PostgreSQL-compatible, not assumed from another vendor.
- Named constraints -- pk_/fk_/ck_ prefixes, never anonymous.
- Migration applies cleanly on the shared training PostgreSQL instance.
- Adapt (never silently ignore) a UUID vs. RAW type mismatch.

Migration excerpt (from the lab guide)

```
CREATE TABLE customer_interaction (  
  interaction_id UUID NOT NULL,  
  customer_id UUID NOT NULL,  
  channel VARCHAR(20) NOT NULL,  
  CONSTRAINT ck_interaction_channel CHECK (channel IN ('PHONE','EMAIL','CHAT'))  
);
```

KEY TAKEAWAY A migration copied from an Oracle-flavored example without adapting types is a named failure mode -- verify every type against PostgreSQL specifically.

Database Schema Review

Ensure the schema is robust, scalable, secure and efficient



1. STRUCTURE & DESIGN

- ✓ Clear and consistent naming conventions
- ✓ Appropriate data types and sizes
- ✓ Proper normalization (1NF, 2NF, 3NF, BCNF)
- ✓ Minimize redundancy and avoid update anomalies
- ✓ Well-defined relationships and cardinality



2. KEYS & CONSTRAINTS

- ✓ Primary keys defined for all tables
- ✓ Foreign keys to enforce referential integrity
- ✓ Unique constraints for business rules
- ✓ NOT NULL constraints where required
- ✓ Check constraints for valid data
- ✓ Default values where appropriate



5. SCALABILITY

- ✓ Design for growth (horizontally & vertically)
- ✓ Avoid hot spots (monotonic keys, counters)
- ✓ Use appropriate sharding strategy
- ✓ Consider archival strategy for old data



6. SECURITY

- ✓ Least privilege access
- ✓ Encrypt sensitive data (at rest & in transit)
- ✓ Avoid storing credentials or PII in plain text
- ✓ Database auditing and monitoring



7. MAINTAINABILITY

- ✓ Consistent naming conventions
- ✓ Well-documented schema and relationships
- ✓ Migration scripts and version control
- ✓ Comments for complex columns and logic



8. RELIABILITY & RECOVERY

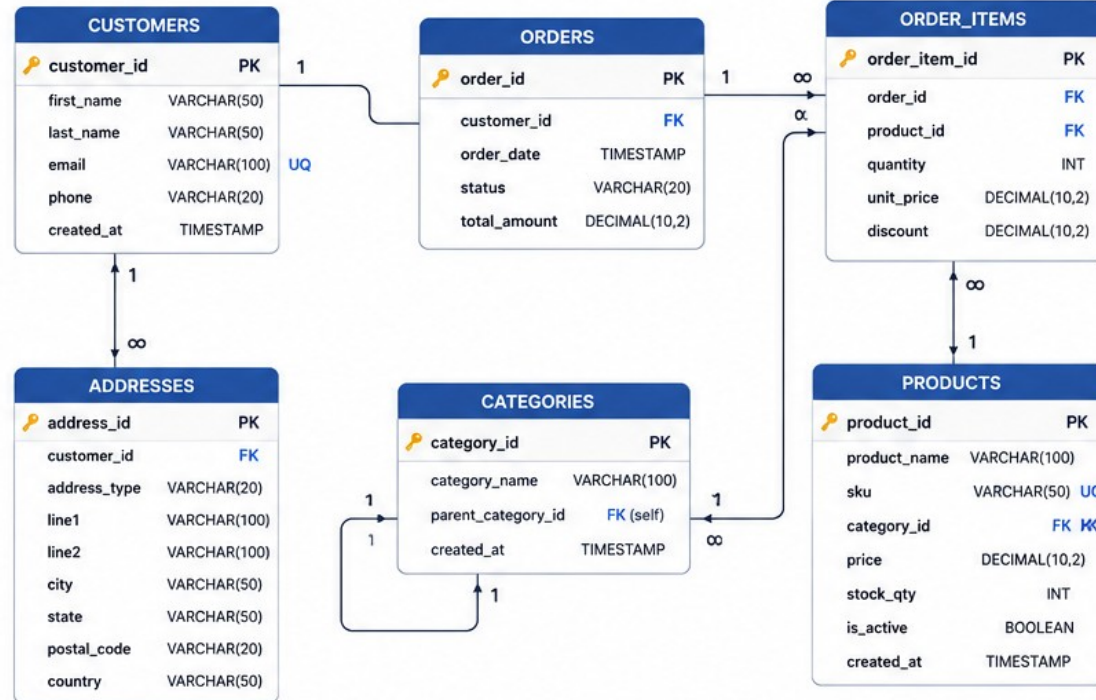
- ✓ Backup strategy (Full + Incremental)
- ✓ Point-in-time recovery
- ✓ Replication and high availability
- ✓ Test restore procedures regularly



9. DOCUMENTATION

- ✓ ER Diagram and data dictionary
- ✓ Column descriptions and constraints
- ✓ Business rules and assumptions
- ✓ Change log and schema history

EXAMPLE SCHEMA (E-COMMERCE)



3. DATA QUALITY

- ✓ Data types match business requirements
- ✓ Consistent formats (e.g., dates, phone, email)
- ✓ Validation rules at the database level
- ✓ No orphan records
- ✓ Consider domain tables for reference data



4. PERFORMANCE

- ✓ Indexes on PK, FK and search columns
- ✓ Avoid over-indexing
- ✓ Review query patterns and bottlenecks
- ✓ Partition large tables if needed
- ✓ Consider materialized views / caching

REVIEW CHECKLIST



Design Sound



Data Accurate



Constraints Complete



Indexes Optimal



Performance Validated



Security Hardened



Scalability Ready



Well Documented



Maintainable & Supportable

ENTITY RELATIONSHIPS REVIEW

Customer owns its interactions -- get relationship ownership and cardinality right before writing a single JPA annotation.

RELATIONSHIP SHAPE

One customer has many interactions (one-to-many).

Interaction is the owning side -- it holds the foreign key.

Customer never cascades a delete onto historical interactions by default.

The relationship direction matches the timeline query's access pattern.

REVIEW DISCIPLINE

- Confirm cardinality on paper before annotating an entity.
- Document ownership explicitly -- do not leave it implicit.
- A missing foreign key means an orphaned interaction row is possible.
- Relationship choices must match whatever Module 48's ADR decided.

Relationship note (concept)

```
// Interaction owns the FK to Customer (interaction.customer_id)
// Customer does NOT hold a @OneToMany collection unless the
// timeline query specifically needs it loaded that way
```

KEY TAKEAWAY An unexamined cascade delete on Customer is exactly the kind of accidental data loss this review exists to catch before it reaches production code.

Entity Relationships Review

Ensure relationships accurately model business rules and support data integrity

Enlationship Types

One-to-One (1:1)



Example: User ↔ UserProfile

One-to-Many (1:N)



Example: Customer → Orders

Many-to-One (N:1)



Example: Orders → Customer

Many-to-Many (M:N)

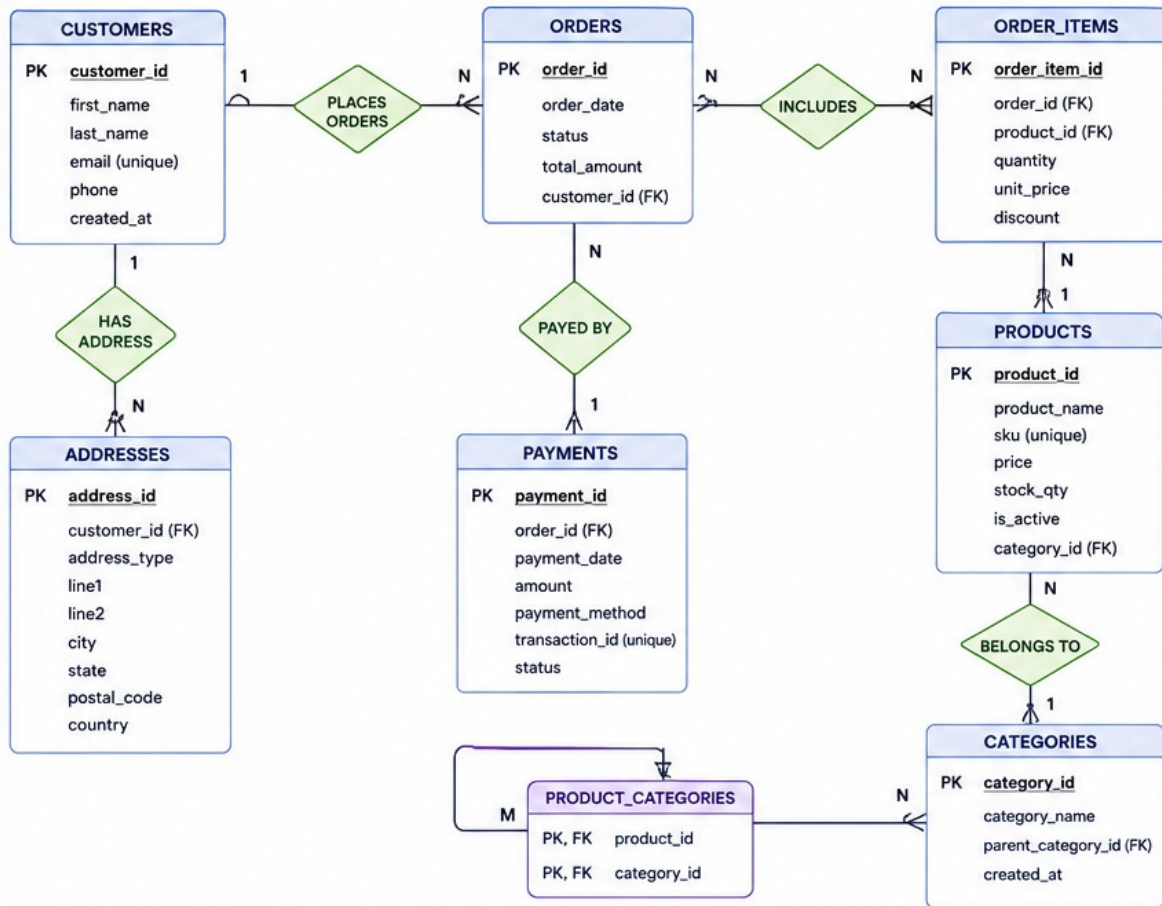


Example: Products ↔ Categories
(via ProductCategories)

Legend



E-COMMERCE ER DIAGRAM (EXAMPLE)



Review Checklist



Business Alignment

- Do relationships reflect business rules?
- Are entities properly associated?



Cardinality

- Are cardinalities correct?
- Are optional vs mandatory relationships defined correctly?



Referential Integrity

- Are foreign keys defined appropriately?
- Are cascade rules appropriate?



Normalization

- Are entities normalized (1NF, 2NF, 3NF)?
- Are repeating groups eliminated?



Performance Impact

- Are relationships optimized for queries?
- Are indexing needs identified?



Documentation

- Are relationships documented?
- Are data dictionary entries updated?

Best Practices



Keep It Simple

Model only what is needed.



Use Consistent Naming

Follow naming conventions for entities, attributes, and relationships.



Avoid Redundancy

Eliminate duplicate data through proper relationships.



Validate with Use Cases

Test relationships against real business scenarios.



Plan for Growth

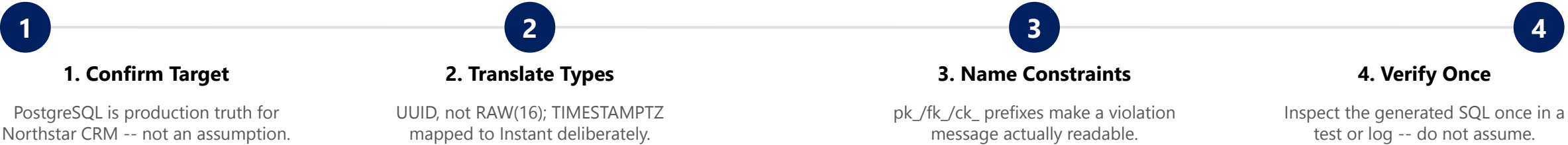
Design relationships that can scale with future requirements.

DATA INTEGRITY STRATEGY (POSTGRESQL VS. ORACLE)

Northstar CRM targets PostgreSQL in production -- know exactly where Oracle habits from earlier weeks do and do not carry over.

Aspect	PostgreSQL (this module)	Oracle (comparison point)
Identifier type	UUID native type or gen_random_uuid()	RAW(16) with SYS_GUID()
Timestamps	TIMESTAMP(6) WITH TIME ZONE	TIMESTAMP WITH TIME ZONE (similar)
Check constraints	CHECK (...) -- same standard syntax	CHECK (...) -- same standard syntax
Sequences/identity	GENERATED ALWAYS AS IDENTITY or sequence	Sequence + trigger, historically
Case sensitivity	Unquoted identifiers fold to lowercase	Unquoted identifiers fold to uppercase

INTEGRITY DISCIPLINE (4 STEPS)



KEY TAKEAWAY A PostgreSQL timezone skew against a Java Instant is a named troubleshooting item -- align TIMESTAMPTZ and Instant deliberately, not by trial and error.

Data Integrity Strategy

Ensure accurate, consistent, and reliable data across its lifecycle

CORE PRINCIPLES



Accuracy

Data correctly represents real-world entities.



Consistency

Data is uniform across systems and over time.



Validity

Data conforms to business rules and constraints.



Completeness

All required data is present and non-redundant.



Timeliness

Data is up-to-date and available when needed.



Integrity by Design

Build controls into processes, systems and culture.

DATA INTEGRITY CONTROLS

1. PREVENT



- Data standards & definitions
- Input validation
- Referential integrity (FKs)
- Business rules & constraints

2. DETECT



- Data quality checks
- Reconciliation
- Exception monitoring
- Anomaly detection

3. CORRECT



- Data cleansing
- Standardization
- Deduplication
- Master data management



RELIABLE DATA

- Trusted for decision making
- Supports analytics & reporting
- Drives business value

4. PROTECT



- Access controls & RBAC
- Data encryption (at rest/in transit)
- Audit logging
- Data masking

5. GOVERN



- Data ownership
- Policies & procedures
- Data stewardship
- Compliance & regulations

6. MONITOR & IMPROVE



- KPIs & dashboards
- Continuous monitoring
- Root cause analysis
- Process improvement

DATA LIFECYCLE INTEGRITY



COLLECT

- Validate at source
- Capture once
- Standard formats



STORE

- Enforce constraints
- Maintain relationships
- Secure storage



PROCESS

- Transform accurately
- Maintain consistency
- Track lineage



SHARE

- Controlled access
- Consistent definitions
- Data agreements



USE

- Quality for purpose
- Verified & trusted
- Timely availability



ARCHIVE / DELETE

- Retention policies
- Secure archival
- Safe disposal

BEST PRACTICES



Establish clear data standards



Automate validation and monitoring



Define ownership and accountability



Train and empower stakeholders



Review and audit regularly



Continuously measure and improve

TRY IT YOURSELF -- EXERCISE 3: SKETCH PERSISTENCE CHANGES

Reinforces: identifying entities, query needs, and a UI-to-database proof plan before writing a migration.

STEPS (notes/lab50-persistence-sketch.md)

Step	What To Do
1 -- Entities	Customer, Interaction (adapt to your schema) -- key fields only.
2 -- Check the Reference	Schema changes go through migrations -- never manual prod edits.
3 -- Query Needs	List queries: search by name/id; interactions by customer, ordered by time.
4 -- Evidence	Plan to prove the UI write appears in PostgreSQL for CUS-1001.

Persistence sketch

```
Entities: Customer, Interaction.  Queries: search, timeline-by-customer.  
Evidence: UI create -> SELECT by customer_id for CUS-1001.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 3 before continuing: exercises/exercise-03-persistence-sketch.md (workspace: examples/module-50-exercises).

ENTITY MAPPING REVIEW

JPA entity fields must map exactly to migration columns -- confirmed by a real repository test, not assumed by reading the class.

MAPPING CHECKLIST

Every entity field maps to an actual migration column, one to one.

@Version on the version column enables optimistic locking.

Lazy fetch preferred; an explicit timeline query avoids N+1.

No cascade type applied without a written reason.

VERIFY BY TEST

- Run a repository test loading Amina's interactions for real.
- LazyInitializationException means a transactional-boundary problem, not a config toggle to hide it.
- Confirm no accidental cascade delete of Customer.
- Record mapping notes in the demo doc, not tribal knowledge.

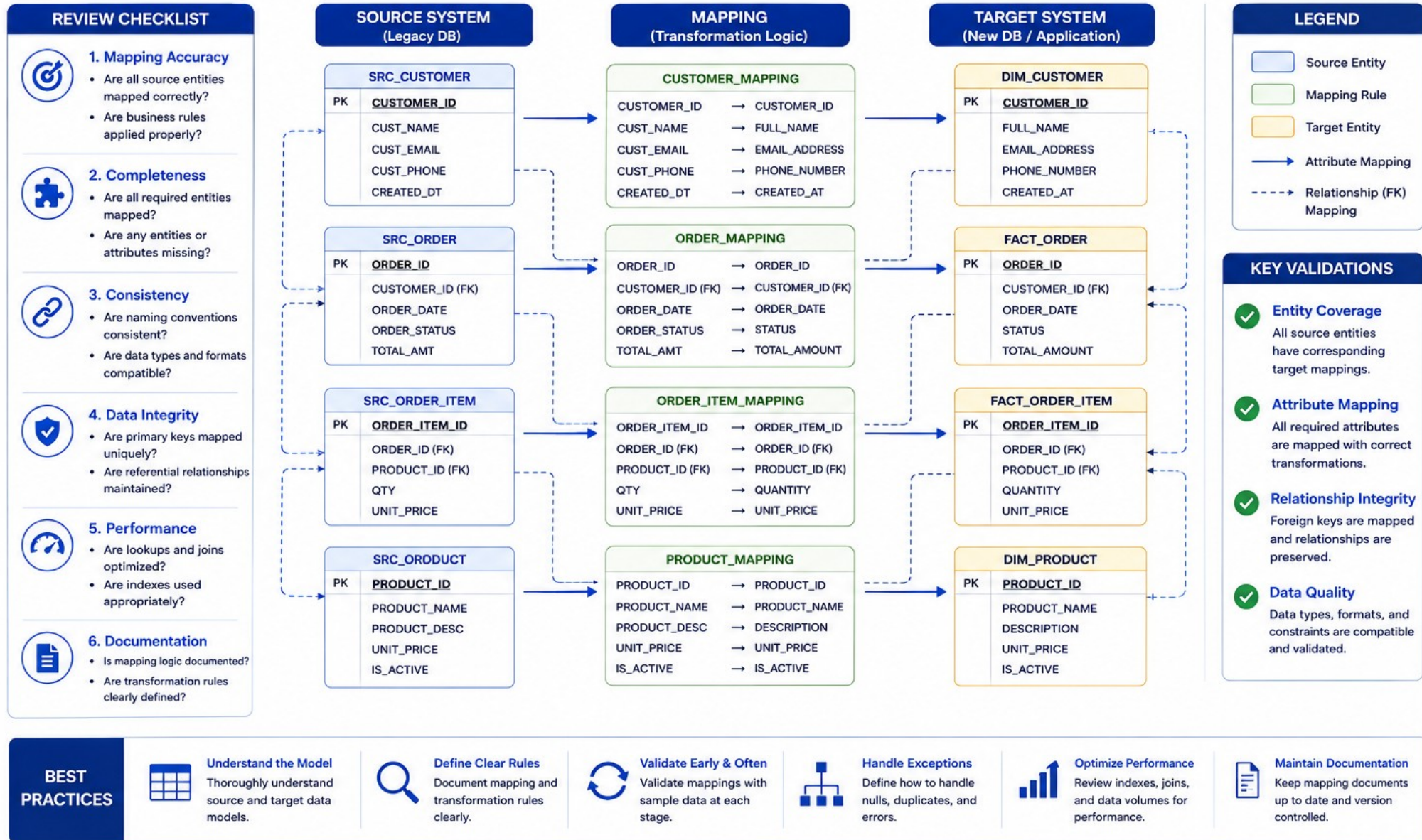
Entity mapping (concept)

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "customer_id")
private Customer customer;
@Version private int version;
```

KEY TAKEAWAY Open Session in View used silently as a default, with no ADR behind it, is explicitly flagged rather than accepted as a fix for LazyInitializationException.

Entity Mapping Review

Ensure accurate, consistent, and complete mapping between source and target entities



REPOSITORY INTEGRATION REVIEW

Spring Data JPA repositories serve exactly the queries this journey needs -- named by business intent, not generic CRUD guessing.

REPOSITORY METHODS

findByCustomerIdOrderByCreatedAtDesc for the timeline read.

Customer lookup by id, reused across search and profile.

Repository stays persistence-only -- no HTTP or DTO concerns.

Seed Amina and Ravi deterministically for repository tests.

INTEGRATION DISCIPLINE

- Repository methods are named for the business need they serve.
- No repository method returns an entity directly to a controller.
- Avoid N+1 by matching the query shape to the actual UI access pattern.
- Repository tests run against the real training PostgreSQL, not a guess.

Repository method (excerpt)

```
List<Interaction> findByCustomerIdOrderByCreatedAtDesc(UUID customerId);
```

KEY TAKEAWAY A repository test that never actually runs against PostgreSQL only proves the method compiles -- not that the query is correct.



PERSISTENCE WORKFLOW (SPRING DATA JPA)

One create action, traced from the browser through JPA to a durable PostgreSQL row that survives a restart.

Term	Meaning
EntityManager / Repository	Spring Data JPA's abstraction over persistence operations.
Transaction boundary	Owned by the service layer -- the same @Transactional pattern from Module 49.
Flush / commit	When pending changes actually reach the database as a durable write.
Restart durability	The row must still be there after the API process restarts.

PERSISTENCE WORKFLOW HABITS



Migration First

Schema exists before the entity maps to it -- never the reverse.



Mapping Second

Entity fields verified against real migration columns.



Repository Third

Business-named queries, tested against real PostgreSQL.

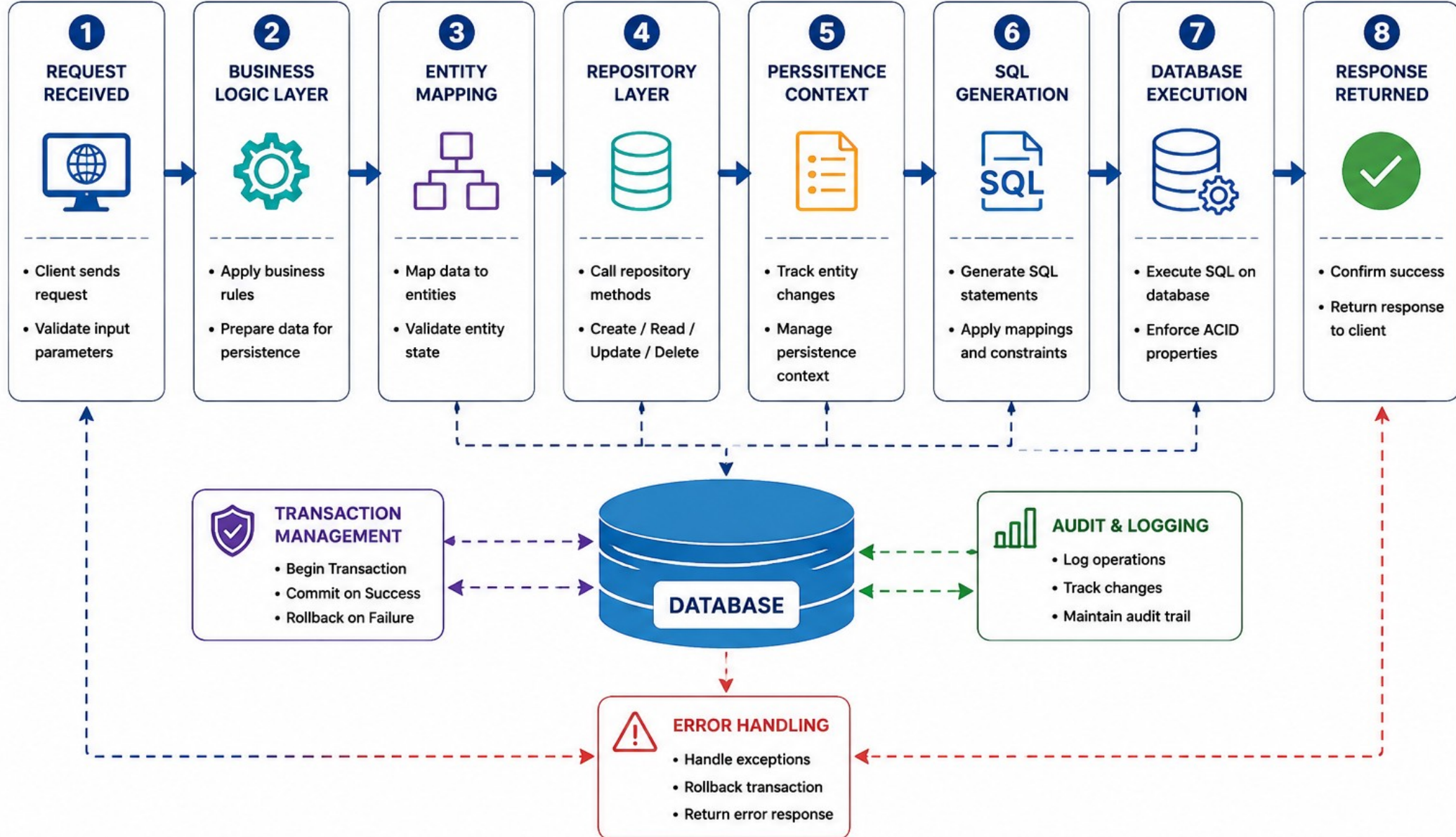


Restart Proof Last

Stop and restart the API; the row for CUS-1001 must remain.

KEY TAKEAWAY UI success with no matching SQL row, caused by the wrong datasource or an uncommitted transaction, is this module's single most graded failure mode.

PERSISTENCE WORKFLOW



FRONTEND TESTING STRATEGY

Component tests isolate rendering and validation logic -- fast, no network, no real backend needed.

WHAT COMPONENT TESTS COVER

InteractionForm validation -- required fields, max length.

InteractionTimeline rendering -- populated and empty states.

Error-state rendering for 400/401/404 responses.

Submit-button disabled state while a request is in flight.

TEST DISCIPLINE

- Use accessible queries (getByLabelText, getByRole), not brittle CSS selectors.
- Mock the typed client, not fetch directly -- test the component's behavior.
- Assert visible text/roles, not internal component state.
- Isolate test data -- do not depend on a shared, mutable fixture.

Component test (concept)

```
render(<InteractionForm customerId={id} onSave={jest.fn()} />);  
fireEvent.click(screen.getByRole('button', { name: /save/i }));  
expect(await screen.findByRole('alert')).toHaveTextContent(/required/i);
```

KEY TAKEAWAY A test coupled to a CSS class name breaks the moment a stylesheet changes -- accessible selectors are the same discipline this module's a11y rules already require.

FRONTEND TESTING STRATEGY



GOALS



Ensure Quality and Reliability



Prevent Regressions



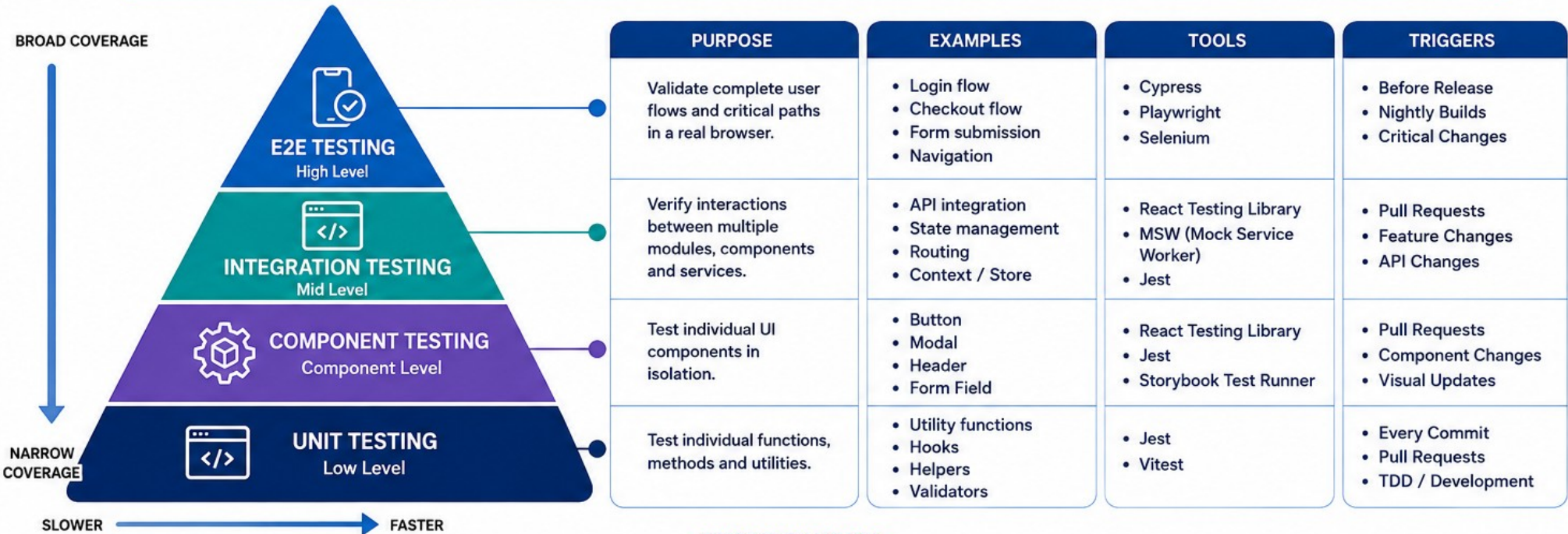
Improve User Experience



Increase Confidence for Releases



Enable Maintainable and Scalable Tests



TESTING ENABLERS



Test Automation First



Reusable Test Utilities & Helpers



Reliable Test Data & Mocks



Continuous Integration & Feedback



Test Reporting & Dashboards



Flaky Test Management



Maintainable Test Suite

STRATEGY WORKFLOW



BEST PRACTICES



Follow Testing Pyramid



Write Clear & Maintainable Tests



Keep Tests Fast & Reliable



Maintain High Test Coverage for Critical Paths



Review & Refactor Tests Regularly

SELENIUM UI TESTING REVIEW

One real, critical-path journey through a real browser -- search Amina, save an interaction, see it on the timeline.

THE CRITICAL PATH

Search for Amina (CUS-1001) in a real, running browser.
Open her profile and timeline.
Submit a new interaction through the real form.
Assert the new interaction's text appears on the timeline.

E2E DISCIPLINE

- Stable selectors -- getByLabel/roles, never a brittle CSS path.
- Seed fixtures in setup -- do not depend on prior test-run state.
- Screenshots on failure only -- not on every single run.
- An instructor-approved substitute is acceptable if the E2E environment is unavailable.

Critical-path outline (concept)

```
driver.findElement(byLabel("Search customers")).sendKeys("Amina");  
// ...select result, fill form, submit...  
assertThat(timelineText()).contains("Resolved login question");
```

KEY TAKEAWAY A Selenium test coupled to a fragile CSS class is a maintenance trap -- the same accessible-selector discipline from component tests applies here too.

SELENIUM UI TESTING REVIEW



PURPOSE

Validate UI functionality, user interactions, and end-to-end scenarios across browsers and platforms.

SELENIUM ARCHITECTURE



TYPES OF UI TESTS



Functional Testing
Validate UI functionality against requirements.



Regression Testing
Ensure new changes do not break existing functionality.



Cross-Browser Testing
Verify application across different browsers.



Responsive Testing
Ensure UI works on different devices and screen sizes.

CORE FEATURES



Browser Automation



Element Interaction (click, type, select, etc.)



Waits and Synchronization



Alerts, Frames & Windows Handling



Test Data Handling



Screenshots & Logs



Assertions & Validations

LOCATORS



ID
Fastest and most reliable.



Name
Uses element name attribute.



XPath
Powerful and flexible.



CSS Selector
Fast and widely used.



Link Text
Find links by visible text.



Class Name, Tag Name
Additional locator strategies.

BEST PRACTICES



Use explicit waits, avoid hard waits.



Use reliable locators.



Keep tests independent.



Follow Page Object Model (POM).



Reuse code and maintain scripts.



Handle exceptions and take logs.



Run tests in different environments.

SELENIUM TESTING WORKFLOW



Requirement Analysis

Understand features and test scenarios.



Test Planning

Define scope, test cases, data and environment.



Test Script Development

Write automation scripts using Selenium APIs.



Test Execution

Run tests on browsers and environments.



Test Results

Capture results, logs, and screenshots.



Defect Reporting

Log and track defects in defect tracker.



Re-Execution

Re-run tests and verify defect fixes.



Test Closure

Analyze results and close the cycle.

TRY IT YOURSELF -- EXERCISE 5: UI VERIFICATION EVIDENCE PLAN

Reinforces: planning component/E2E evidence and screenshot names without claiming full coverage that does not exist yet.

STEPS (notes/lab50-verification-plan.md)

Step	What To Do
1 -- Cases	Search hit, profile render, create interaction, error banner on API failure.
2 -- Tools	Note the assigned approach -- component test and/or E2E, per instructor.
3 -- Screenshots	Name evidence files under notes/screenshots/lab-50/.
4 -- Data Proof	Include a DB/API proof step right after any UI write.

Verification plan skeleton

```
Cases: search hit, profile, create, error banner.  
Evidence: notes/screenshots/lab-50/*.png   Proof: SQL SELECT after each write.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 5 before continuing: exercises/exercise-05-verification-plan.md (workspace: examples/module-50-exercises).

TRY IT YOURSELF -- EXERCISE 6: DOCUMENT END-TO-END DATA FLOW

Reinforces: writing a numbered UI-to-database trace for one action, with the failure-surfacing point marked explicitly.

STEPS (notes/lab50-data-flow-note.md)

Step	What To Do
1 -- Pick Action	Add an interaction on CUS-1001 with correlation lab-request-001.
2 -- Sequence	Numbered steps across frontend, controller, service, repository, DB row.
3 -- Failure Point	Mark where validation or a DB failure actually surfaces in the UI.
4 -- Scope	Notes only -- the full journey is built and proven during Lab 50.

Data-flow trace skeleton

```
1. Form submit -> 2. Typed client POST -> 3. Controller -> 4. Service (@Transactional)
-> 5. Repository save -> 6. 201 -> 7. Timeline refetch -> 8. Row visible in PostgreSQL.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 6 before continuing: exercises/exercise-06-data-flow-note.md (workspace: examples/module-50-exercises).

LAB OVERVIEW -- NORTHSTAR CRM UI→POSTGRESQL JOURNEY

Complete a usable React CRM journey backed by Spring Data JPA and PostgreSQL, proving validation, persistence, and accessibility end to end.

BUSINESS SCENARIO

- Agents must complete the customer-facing workflow for Amina and Ravi. Leadership freezes: no Lab 50 pass without UI proof, PostgreSQL row proof, accessibility basics, and at least one automated UI/component check on the critical path.
- You own the frontend-and-persistence gate for search, profile, timeline, and create interaction.
- A peer must be able to reproduce every result from docs/frontend-persistence-demo.md alone, without verbal coaching.

LEARNING OBJECTIVES

- Build typed React service calls against Module 49's API contracts.
- Implement accessible forms with labels, focus management, and alerts.
- Complete JPA and PostgreSQL mappings with owned relationships.
- Create repeatable, versioned migrations.
- Handle loading, empty, success, invalid, unauthorized, and outage states.
- Verify UI-to-database flow with restart durability, plus critical-path automation.

WHAT YOU BUILD

- customer-management-platform/frontend/: CustomerSearch, CustomerProfile, InteractionTimeline, InteractionForm, a typed API client, plus backend migration and JPA mapping changes, tests, and docs/frontend-persistence-demo.md.

KEY TAKEAWAY Fixtures CUS-1001 (Amina), CUS-1002 (Ravi), and CUS-9999 (not-found) are synthetic -- never real customer PII. Duration: about 45 minutes with starters, up to 6-8 hours full path. Advanced capstone difficulty.

LAB TASKS AND DELIVERABLES

lab50-crm -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Choose the end-to-end journey and document the storyboard.
2	Complete the PostgreSQL schema with named constraints and indexes.
3	Review JPA mapping: ownership, fetch strategy, cascade, @Version.
4	Create the typed API client with cancellation/stale-response protection.
5	Build the accessible UI: search, profile, timeline, form.
6	Handle server state: loading/empty/success/invalid/unauthorized/outage.
7	Verify persistence: UI create, API read, SQL row, restart durability.
8	Automate the critical path: component tests + one Selenium journey.

EXPECTED DELIVERABLES

- React components for search, profile, timeline, and interaction form
- Typed API client
- JPA mapping changes as required
- PostgreSQL migration scripts
- Component and UI/E2E tests, plus docs/frontend-persistence-demo.md

RUBRIC (100 MARKS)

- Environment/Structure 10 * Core Impl 30 * Integration/Config 15 * Failure Handling 15 * Verification 10 * Security/Production Awareness (a11y) 10 * Docs 10

KEY TAKEAWAY Pretty UI without SQL proof loses core marks. Missing labels or keyboard support loses a11y/security marks. DTO drift from Module 49 costs an integration deduction.

MODULE 50 SUMMARY

In this module, we completed the Northstar CRM journey end to end: an accessible React UI backed by proven PostgreSQL persistence.

KEY CONCEPTS COVERED



Connected Journey

Search, profile, timeline, and form as one flow, not isolated components.



Typed API Client

One centralized client, cancellation-safe, correlation header always sent.



Accessibility Graded

Real labels, keyboard order, and role=alert -- not color-only feedback.



PostgreSQL Proof

Versioned migrations, reviewed JPA mappings, restart-durability evidence.



Full State Matrix

Loading, empty, success, invalid, unauthorized, and outage -- all six.



Tests as Evidence

Component tests plus one critical-path Selenium journey.

MODULE FLOW RECAP

1

1. Map Journey

2

2. Build UI + Client

3

3. Wire Persistence

4

4. Handle All States

5

5. Prove + Automate

KEY TAKEAWAY A UI is graded end to end: connected journey, typed client, accessibility, PostgreSQL proof, full state handling, and automated tests -- all six, not just the happy path.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of the typed client, accessibility, and state handling.

1. Where should every network call in the React journey originate?

- A. Directly inside each component's fetch call
- B. A single, centralized typed API client**
- C. A global variable set once at startup
- D. Inline inside the JSX render

3. What actually proves a UI write is persistent, per this module's rule?

- A. A success toast appearing
- B. A SQL row that survives an API restart**
- C. The browser network tab showing 201
- D. The developer's memory that it worked once

2. What must replace placeholder text as a form field's label?

- A. A tooltip
- B. A real `<label htmlFor>` element**
- C. A color change on focus
- D. Nothing -- placeholders are sufficient

4. What should the UI do when the API returns a 400 Problem Details response?

- A. Show a success message anyway
- B. Show field-level errors via `role="alert"`**
- C. Silently ignore the response
- D. Retry the same request automatically forever

KEY TAKEAWAY Centralize network calls in one typed client; real labels beat placeholders; restart-surviving SQL rows are the only persistence proof; 400s map to accessible field errors.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on persistence discipline, fixtures, testing, and the merge gate.

5. Which fixture ID exercises the not-found empty state in Lab 50?

- A. CUS-1001
- B. CUS-1002
- C. CUS-9999**
- D. lab-request-001

7. Per this module's testing scope, what does the required Selenium journey cover?

- A. Every possible UI state exhaustively
- B. One real critical path: search, save, verify the timeline**
- C. Only the loading spinner's CSS animation
- D. Backend unit tests exclusively

6. What must happen before a JPA entity can safely map to a database column?

- A. Nothing -- Hibernate infers it automatically and safely
- B. A versioned migration must already define that column**
- C. The frontend must call the API once
- D. The column name must match a Java keyword

8. What is required before Lab 50 can be marked a pass, per this module's rule?

- A. A single successful screenshot
- B. UI proof, PostgreSQL row proof, accessibility basics, and one automated check**
- C. Only a green npm build
- D. Instructor sign-off with no evidence

KEY TAKEAWAY CUS-9999 is the not-found fixture; migrations precede entity mapping; Selenium covers one critical path; a pass requires all four forms of evidence together.

MODULE 50 COMPLETE

Capstone Frontend and Persistence

You can now build an accessible React journey -- search, profile, timeline, and form -- typed against a real API, backed by PostgreSQL persistence you can prove survives a restart, with component and Selenium tests covering the critical path.